

# **Разработка локального LLM сервера и внедрение AI-агента в рабочую среду**

Специальность: 09.02.06 Сетевое и системное администрирование

Выполнил:  
Беловодский П.В.,  
студент группы  
ССА-25-20  
Руководитель:  
Тимохов А.А.

# Актуальность

Проблема: облачные AI-сервисы (Copilot, ChatGPT) создают риски утечки кода и требуют постоянных платежей

Тренды:

- 84% разработчиков используют AI (2025)
- 29% кода на GitHub создано с помощью AI (2024)

Решение — локальный LLM-сервер:

- Код остаётся внутри организации
- Работа без интернета
- Отсутствие абонентской платы

# Объект и предмет

## **Объект исследования:**

Процесс организации безопасной локальной инфраструктуры для использования LLM в разработке ПО

## **Предмет исследования:**

Методы развертывания, администрирования и интеграции локального LLM-сервера в рабочую среду IT-специалистов

# Цели и Задачи

## Цель работы:

Разработать и экспериментально подтвердить работоспособность локального LLM-сервера, интегрированного со средой разработки Code OSS

## Задачи:

1. Проанализировать существующие решения для локального развертывания LLM
2. Определить аппаратные требования к серверной платформе
3. Разработать конфигурацию сервера и выполнить настройку ПО
4. Реализовать интеграцию с Code OSS
5. Проверить работоспособность решения
6. Сформулировать рекомендации по администрированию

# Выбор решения

| Критерий             | Ollama        | llama.cpp                 |
|----------------------|---------------|---------------------------|
| Параллельные сессии  | Нет (очередь) | Да (настраиваемое кол-во) |
| Управление ресурсами | Нет           | Да (--threads)            |
| Веб-интерфейс        | Нет           | Да                        |
| Логирование          | Минимальное   | Детальное                 |
| Простота настройки   | Да            | Нет                       |

## Аппаратные требования к серверу

**Факторы, влияющие на требования:** размер модели и тип квантования, количество параллельных сессий, размер контекстного окна

Требования для 5–10 разработчиков (модель 7B–13B)

| Компонент  | Минимум   | Рекомендуется |
|------------|-----------|---------------|
| CPU        | 8 ядер    | 16 ядер       |
| RAM        | 16 ГБ     | 64 ГБ         |
| GPU        | 8 ГБ VRAM | 16 ГБ VRAM    |
| Накопитель | 64 ГБ SSD | 128 ГБ SSD    |
| Сеть       | 1 Гбит/с  | 1 Гбит/с      |

# Разработка конфигурации сервера

ПО: Arch Linux, llama.cpp (скомпилирован из исходников)

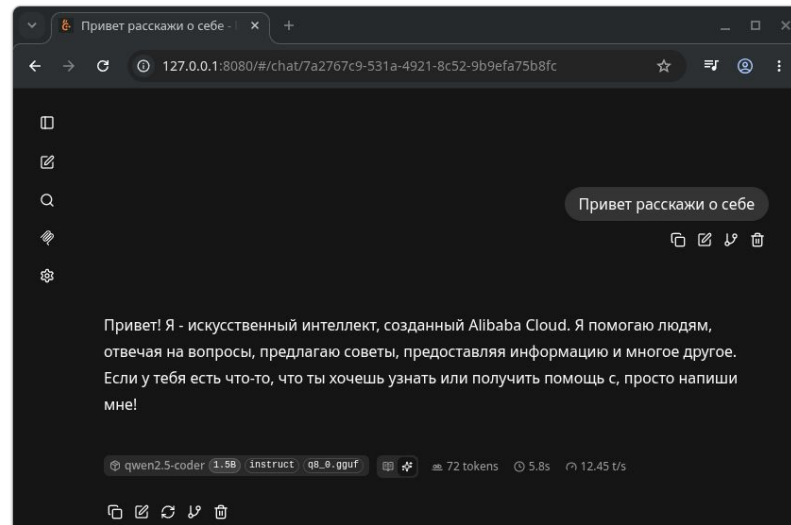
Модель: Qwen2.5-Coder-1.5B (GGUF, Q8\_0, ~1,8 ГБ)

Результат: веб-интерфейс доступен по адресу

`http://127.0.0.1:8080`, модель успешно отвечает

на запросы

```
17 # Запуск сервера
18 nohup /home/chitak/LLAMA/llama.cpp/build-pc/bin/llama-server \
19   -m "$MODEL_PATH" \
20   --host "$HOST" \
21   --port "$PORT" \
22   -c 4096 \
23   -ngl 33 \
24   --threads 4 \
25   --parallel 2 \
26   > "$LOG_FILE" 2>&1 &
27
```



# Интеграция с Code OSS

**Инструмент интеграции:** Continue

(открытое расширение для VS Code / Code OSS)

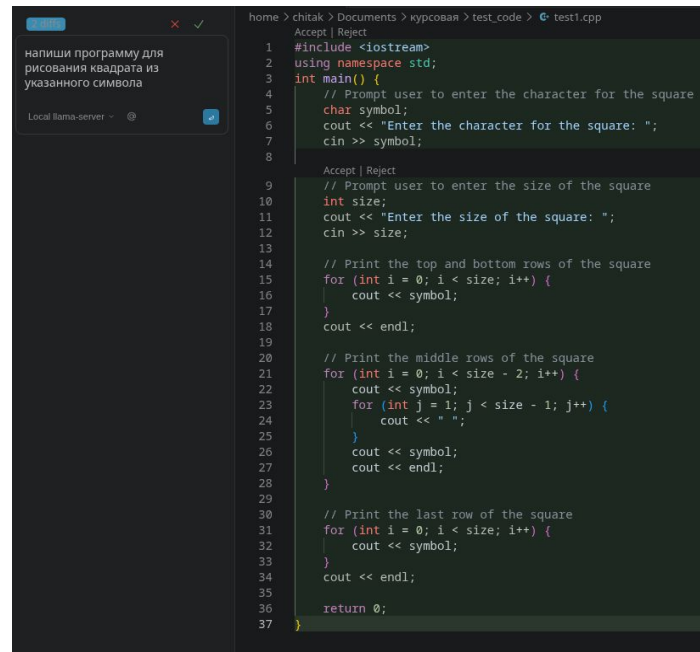
**Конфигурация:** ~/.continue/config.yaml на рисунке

```
home > chitak > .continue > ! config.yaml
View Continue Reference | Download YAML extension for Intellisense
1  name: Local Config
2  version: 1.0.0
3  schema: v1
4
5  models:
6    - name: Local llama-server
7      provider: llama.cpp
8      model: qwen2.5-coder-1.5b-instruct-q8_0.gguf # имя модели
9      apiBase: http://127.0.0.1:8080 # адрес сервера
10     contextLength: 8192
11     roles:
12       - chat
13       - edit
14       - apply
15
16   tabAutocompleteModel:
17     provider: llama.cpp
18     model: qwen2.5-coder-1.5b-instruct-q8_0.gguf
19     apiBase: http://127.0.0.1:8080
20     contextLength: 4096
21
22   allowAnonymousTelemetry: false
```

# Результат интеграции

**Результаты:** подключение установлено, модель доступна в интерфейсе Continue, сгенерирован рабочий код (программа рисования квадрата)

```
chitak@X99:~/Documents/курсовая/test_code$ g++ -Wall -Wextra test1.cpp -o test1-bin
chitak@X99:~/Documents/курсовая/test_code$ ./test1-bin
Enter the character for the square: #
Enter the size of the square: 16
#####
#               #
#               #
#               #
#               #
#               #
#               #
#               #
#               #
#               #
#               #
#               #
#               #
#               #
#               #
#               #
#####
chitak@X99:~/Documents/курсовая/test_code$
```



```
home > chitak > Documents > курсовая > test_code > test1.cpp
Accept | Reject
напиши программу для
рисования квадрата из
указанного символа
Local llama-server - @
Accept | Reject
1 // Prompt user to enter the character for the square
2 #include <iostream>
3 using namespace std;
4 int main() {
5     char symbol;
6     cout << "Enter the character for the square: ";
7     cin >> symbol;
8
9     // Prompt user to enter the size of the square
10    int size;
11    cout << "Enter the size of the square: ";
12    cin >> size;
13
14    // Print the top and bottom rows of the square
15    for (int i = 0; i < size; i++) {
16        cout << symbol;
17    }
18    cout << endl;
19
20    // Print the middle rows of the square
21    for (int i = 0; i < size - 2; i++) {
22        cout << symbol;
23        for (int j = 1; j < size - 1; j++) {
24            cout << " ";
25        }
26        cout << symbol;
27        cout << endl;
28    }
29
30    // Print the last row of the square
31    for (int i = 0; i < size; i++) {
32        cout << symbol;
33    }
34    cout << endl;
35
36    return 0;
37 }
```

## Выводы

В ходе выполнения курсовой работы были выполнены задачи по развёртыванию локального LLM-сервера и его интеграции в рабочую среду разработчика.

Произведен анализ существующих решений (Ollama, llama.cpp), на основе которого обоснован выбор llama.cpp как оптимального инструмента для многопользовательского сценария.

Выполнены расчёты аппаратных требований: для обслуживания 5–10 разработчиков с моделью 7B–13B определены минимальная (16 ГБ ОЗУ, 8 ядер CPU) и рекомендуемая (64 ГБ ОЗУ, 16 ядер CPU, GPU 16 ГБ VRAM) конфигурации.

Результаты работы и сделанные выводы могут быть использованы для внедрения локального AI-ассистента в IT-отделах организаций с высокими требованиями к безопасности и автономности, а также для дальнейшего развития проекта в направлении использования более крупных моделей и GPU-ускорения.